

计算机系统漫游

计算机系统是由硬件和系统软件组成的，它们共同工作来运行应用程序。虽然系统的具体实现方式随着时间不断变化，但是系统内在的概念却没有改变。所有计算机系统都有相似的硬件和软件组件，它们又执行着相似的功能。一些程序员希望深入了解这些组件是如何工作的以及这些组件是如何影响程序的正确性和性能的，以此来提高自身的技能。本书便是为这些读者而写的。

现在就要开始一次有趣的漫游历程了。如果你全力投身学习本书中的概念，完全理解底层计算机系统以及它对应用程序的影响，那么你会步上成为为数不多的“大牛”的道路。

你将会学习一些实践技巧，比如如何避免由计算机表示数字的方式引起的奇怪的数字错误。你将学会怎样通过一些小窍门来优化自己的 C 代码，以充分利用现代处理器和存储器系统的设计。你将了解编译器是如何实现过程调用的，以及如何利用这些知识来避免缓冲区溢出错误带来的安全漏洞，这些弱点给网络和因特网软件带来了巨大的麻烦。你将学会如何识别和避免链接时那些令人讨厌的错误，它们困扰着普通的程序员。你将学会如何编写自己的 Unix shell、自己的动态存储分配包，甚至于自己的 Web 服务器。你会认识并带来的希望和陷阱，这个主题随着单个芯片上集成了多个处理器核变得越来越重要。

在 Kernighan 和 Ritchie 的关于 C 编程语言的经典教材[61]中，他们通过图 1-1 中所示的 hello 程序来向读者介绍 C。尽管 hello 程序非常简单，但是为了让它实现运行，系统的每个主要组成部分都需要协调工作。从某种意义上来说，本书的目的就是要帮助你了解当你在系统上执行 hello 程序时，系统发生了什么以及为什么会这样。

```
code/intro/hello.c
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("hello, world\n");
6      return 0;
7  }
```

code/intro/hello.c

图 1-1 hello 程序(来源: [60])

我们通过跟踪 hello 程序的生命周期来开始对系统的学习——从它被程序员创建开始，到在系统上运行，输出简单的消息，然后终止。我们将沿着这个程序的生命周期，简要地介绍一些逐步出现的关键概念、专业术语和组成部分。后面的章节将围绕这些内容展开。

1.1 信息就是位 + 上下文

hello 程序的生命周期是从一个源程序(或者说源文件)开始的，即程序员通过编辑器创建并保存的文本文件，文件名是 hello.c。源程序实际上就是一个由值 0 和 1 组成的位(又称为比特)序列，8 个位被组织成一组，称为字节。每个字节表示程序中的某些文本字符。

大部分的现代计算机系统都使用 ASCII 标准来表示文本字符,这种方式实际上就是一个唯一的单字节大小的整数值^①来表示每个字符。比如,图 1-2 中给出了 hello.c 程序的 ASCII 码表示。

#	i	n	c	l	u	d	e	SP	<	s	t	d	i	o	.
35	105	110	99	108	117	100	101	32	60	115	116	100	105	111	46
h	>	\n	\n	i	n	t	SP	m	a	i	n	(	)	\n	{
104	62	10	10	105	110	116	32	109	97	105	110	40	41	10	123
\n	SP	SP	SP	SP	p	r	i	n	t	f	(	"	h	e	l
10	32	32	32	32	112	114	105	110	116	102	40	34	104	101	108
l	o	,	SP	w	o	r	l	d	\	n	"	)	;	\n	SP
108	111	44	32	119	111	114	108	100	92	110	34	41	59	10	32
SP	SP	SP	r	e	t	u	r	n	SP	0	;	\n	}	\n	
32	32	32	114	101	116	117	114	110	32	48	59	10	125	10	

图 1-2 hello.c 的 ASCII 文本表示

hello.c 程序是以字节序列的方式储存在文件中的。每个字节都有一个整数值,对应于某些字符。例如,第一个字节的整数值是 35,它对应的就是字符“#”。第二个字节的整数值为 105,它对应的字符是‘i’,依此类推。注意,每个文本行都是以一个看不见的换行符‘\n’来结束的,它所对应的整数值为 10。像 hello.c 这样只由 ASCII 字符构成的文件称为文本文件,所有其他文件都称为二进制文件。

hello.c 的表示方法说明了一个基本思想:系统中所有的信息——包括磁盘文件、内存中的程序、内存中存放的用户数据以及网络上传送的数据,都是由一串比特表示的。区分不同数据对象的唯一方法是我们读到这些数据对象时的上下文。比如,在不同的上下文中,一个同样的字节序列可能表示一个整数、浮点数、字符串或者机器指令。

作为程序员,我们需要了解数字的机器表示方式,因为它们与实际的整数和实数是不同的。它们是对真值的有限近似值,有时候会有意想不到的行为表现。这方面的基本原理将在第 2 章中详细描述。

旁注 C 编程语言的起源

C 语言是贝尔实验室的 Dennis Ritchie 于 1969 年~1973 年间创建的。美国国家标准学会(American National Standards Institute, ANSI)在 1989 年颁布了 ANSI C 的标准,后来 C 语言的标准化成国际标准化组织(International Standards Organization, ISO)的责任。这些标准定义了 C 语言和一系列函数库,即所谓的 C 标准库。Kernighan 和 Ritchie 在他们的经典著作中描述了 ANSI C,这本著作被人们满怀感情地称为“K&R”[61]。用 Ritchie 的话来说[92],C 语言是“古怪的、有缺陷的,但同时也是一个巨大的成功”。为什么会成功呢?

- C 语言与 Unix 操作系统关系密切。C 从一开始就是作为一种用于 Unix 系统的程序语言开发出来的。大部分 Unix 内核(操作系统的核心部分),以及所有支撑工具和函数库都是用 C 语言编写的。20 世纪 70 年代后期到 80 年代初期,Unix 风行于高等院校,许多人开始接触 C 语言并喜欢上它。因为 Unix 几乎全部是用 C 编写的,它可以很方便地移植到新的机器上,这种特点为 C 和 Unix 赢得了更为广泛的支持。

① 有其他编码方式用于表示非英语类语言文本。具体讨论参见 2.1.4 节的旁注。

- C语言小而简单。C语言的设计是由一个人而非一个协会掌控的，因此这是一个简洁明了、没有什么冗赘的设计。K&R这本书用大量的例子和练习描述了完整的C语言及其标准库，而全书不过261页。C语言的简单使它相对而言易于学习，也易于移植到不同的计算机上。

- C语言是为实践目的设计的。C语言是设计用来实现Unix操作系统的。后来，其他人发现能够用这门语言无障碍地编写他们想要的程序。

C语言是系统级编程的首选，同时它也非常适用于应用级程序的编写。然而，它也并非适用于所有的程序员和所有的情况。C语言的指针是造成程序员困惑和程序错误的一个常见原因。同时，C语言还缺乏对非常有用的抽象的显式支持，例如类、对象和异常。像C++和Java这样针对应用级程序的新程序语言解决了这些问题。

1.2 程序被其他程序翻译成不同的格式

hello程序的生命周期是从一个高级C语言程序开始的，因为这种形式能够被人读懂。然而，为了在系统上运行hello.c程序，每条C语句都必须被其他程序转化为一连串的低级机器语言指令。然后这些指令按照一种称为可执行目标程序的格式打好包，并以二进制磁盘文件的形式存放起来。目标程序也称为可执行目标文件。

在Unix系统上，从源文件到目标文件的转化是由编译器驱动程序完成的：

```
linux> gcc -o hello hello.c
```

在这里，GCC编译器驱动程序读取源程序文件hello.c，并把它翻译成一个可执行目标文件hello。这个翻译过程可分为四个阶段完成，如图1-3所示。执行这四个阶段的程序(预处理器、编译器、汇编器和链接器)一起构成了编译系统(compilation system)。

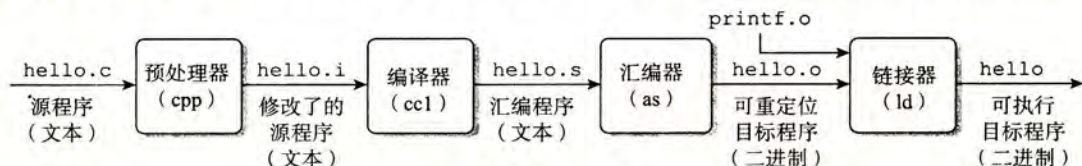


图 1-3 编译系统

- 预处理阶段。预处理器(cpp)根据以字符#开头的命令，修改原始的C程序。比如hello.c中第1行的#include <stdio.h>命令告诉预处理器读取系统头文件stdio.h的内容，并把它直接插入程序文本中。结果就得到了另一个C程序，通常是以.i作为文件扩展名。
- 编译阶段。编译器(cc1)将文本文件hello.i翻译成文本文件hello.s，它包含一个汇编语言程序。该程序包含函数main的定义，如下所示：

```

1  main:
2      subq    $8, %rsp
3      movl    $.LC0, %edi
4      call    puts
5      movl    $0, %eax
6      addq    $8, %rsp
7      ret

```

定义中2~7行的每条语句都以一种文本格式描述了一条低级机器语言指令。汇编语言是非常有用的，因为它为不同高级语言的不同编译器提供了通用的输出语